

Computer Networks

Unit-4 Data Link Layer

Prepared By Dr. Suvarna Sharma

Department of Computer Science and Applications
Dr. Vishwanath Karad MIT World Peace University

Index

1. Functionality of Data-link Layer
2. Types of Errors
3. Redundancy
4. Error Detection
5. Error Correction

Unit 4 : The Data Link Layer: 5 Design Issues—Services provided to the Network Layer Framing—Concept, Methods-Character Count, Flag bytes with Byte Stuffing, Starting & ending Flags with Bit Stuffing and Physical Layer Coding Violations, Flow and Error Control Error detection, code CRC.

Data-link Layer

Data link layer is responsible for converting data stream to signals bit by bit and to send that over the underlying hardware. At the receiving end, Data link layer picks up data from hardware which are in the form of electrical signals, assembles them in a recognizable frame format, and hands over to upper layer.



DATA LINK LAYER DESIGN ISSUES

- Services Provided to the Network Layer
- Framing
- Error Control
- Flow Control

Functionality of Data-link Layer

1. **Framing:** Data-link layer takes packets from Network Layer and encapsulates them into Frames. Then, it sends each frame bit-by-bit on the hardware. At receiver end, data link layer picks up signals from hardware and assembles them into frames.
2. **Addressing:** Data-link layer provides layer-2 hardware addressing mechanism. Hardware address is assumed to be unique on the link. It is encoded into hardware at the time of manufacturing.
3. **Synchronization:** When data frames are sent on the link, both machines must be synchronized in order to transfer to take place.

Functionality of Data-link Layer

4. **Error Control:** Sometimes signals may have encountered problem in transition and the bits are flipped. These errors are detected and attempted to recover actual data bits. It also provides error reporting mechanism to the sender.
5. **Flow Control:** Stations on same link may have different speed or capacity. Data-link layer ensures flow control that enables both machine to exchange data on same speed.
6. **Multi-Access:** When host on the shared link tries to transfer the data, it has a high probability of collision. Data-link layer provides mechanism such as CSMA/CD to equip capability of accessing a shared media among multiple Systems.

SERVICES PROVIDED TO THE NETWORK LAYER

1. Unacknowledged Connectionless Service
2. Acknowledged Connectionless Service
3. Acknowledged Connection-oriented Service (Three phases)
 - Connection establishment
 - data transfer
 - disconnection

SERVICES TO NETWORK LAYER

- **Unacknowledged connectionless service**
 - No connection required and without acknowledgement for data frames
 - Appropriate for low error rate and real-time traffic
 - Error recovery is up to higher layer

SERVICES TO NETWORK LAYER

- **Acknowledged connectionless service**
 - No connection required but each frame is individually acknowledged
 - Useful for unreliable channel, such as wireless systems.
 - Transport layer may do *message recovery* but is more expensive than *frame recovery* at data link layer

SERVICES TO NETWORK LAYER

- **Acknowledged connection-oriented service**
 - Guarantee error-free and in sequence delivery of data frames
 - Consists of three phases
 - **Connection set up** (variables and buffers initialization)
 - **Data frame transmission**
 - **Connection termination** (free of variable and buffers)

FRAMING

*The data link layer needs to pack bits into **frames**, so that each frame is distinguishable from another. Our postal system practices a type of framing. The simple act of inserting a letter into an envelope separates one piece of information from another; the envelope serves as the delimiter.*

Topics discussed in this section:

Fixed-Size Framing

Variable-Size Framing

FRAMING

Break bit stream from physical layer into frames for error detection and recovery.

Four framing methods

1. Character count
2. Flag bytes with byte (character) stuffing
3. Starting and ending flags, with bit stuffing
4. Physical layer coding violations

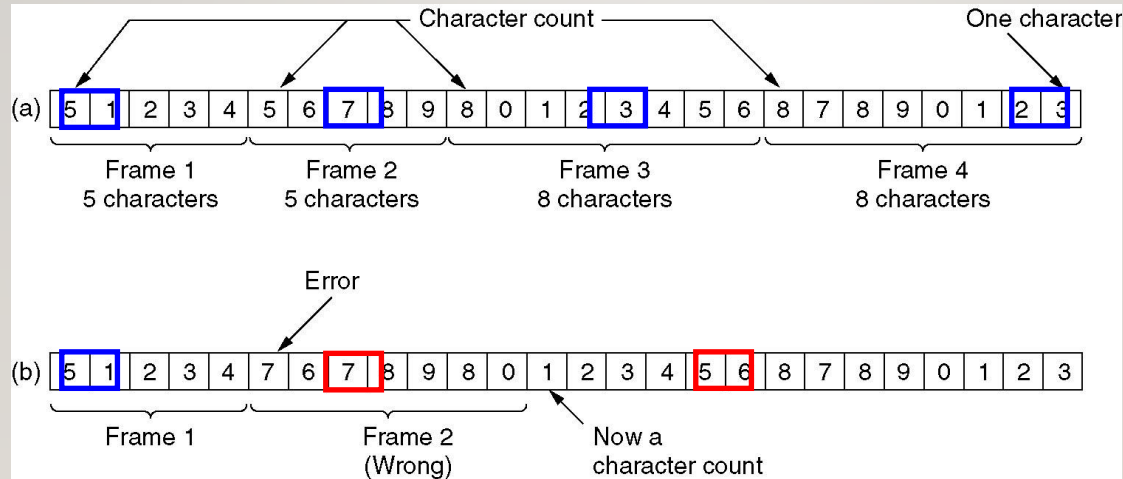
Framing Approaches (1)

Character Count

- Indicate the frame boundary by frame length
- Once an error, frame boundary cannot be recognized and thus recovery is impossible.

Framing Approaches (1)

A character stream. (a) Without errors. (b) With one error.



Framing Approaches (2)

Flag bytes with byte stuffing (character stuffing)

- Delimit the frame by flag bytes
- Prevent frame boundary from appearing at the data content by character stuffing.
- Character Stuffing: inserting ESC ahead of accidental flag byte within the data content.

Figure *A frame in a character-oriented protocol*

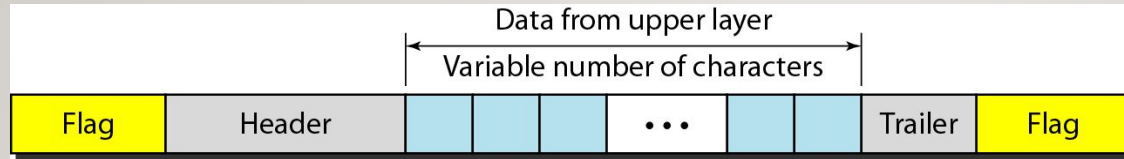


Figure 11.2 *Byte stuffing and unstuffing*

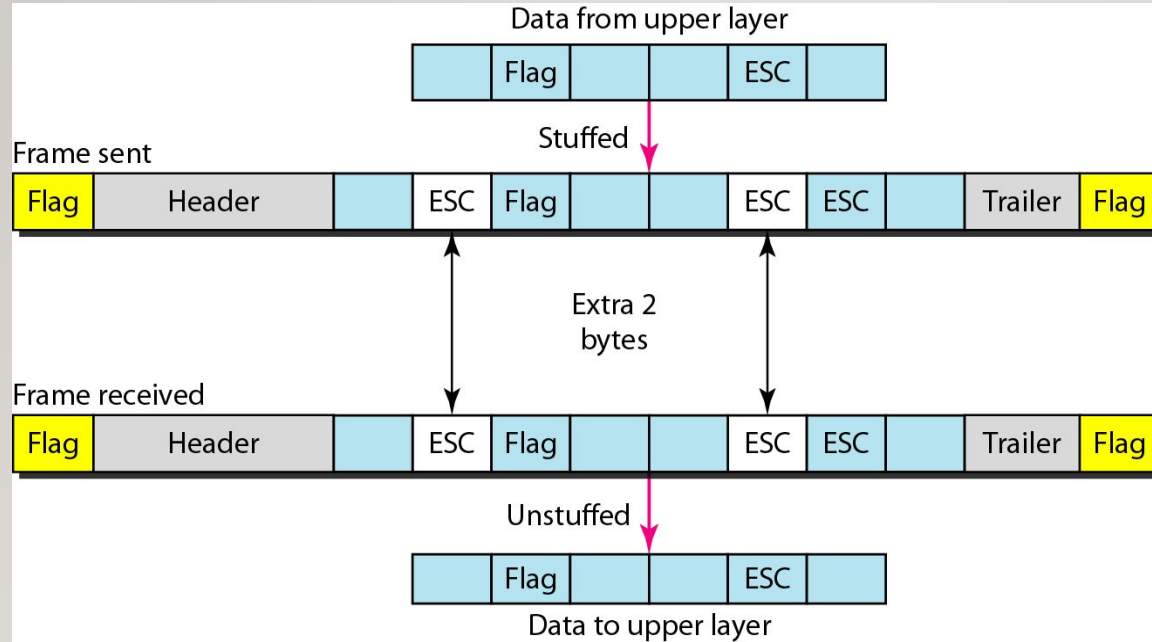
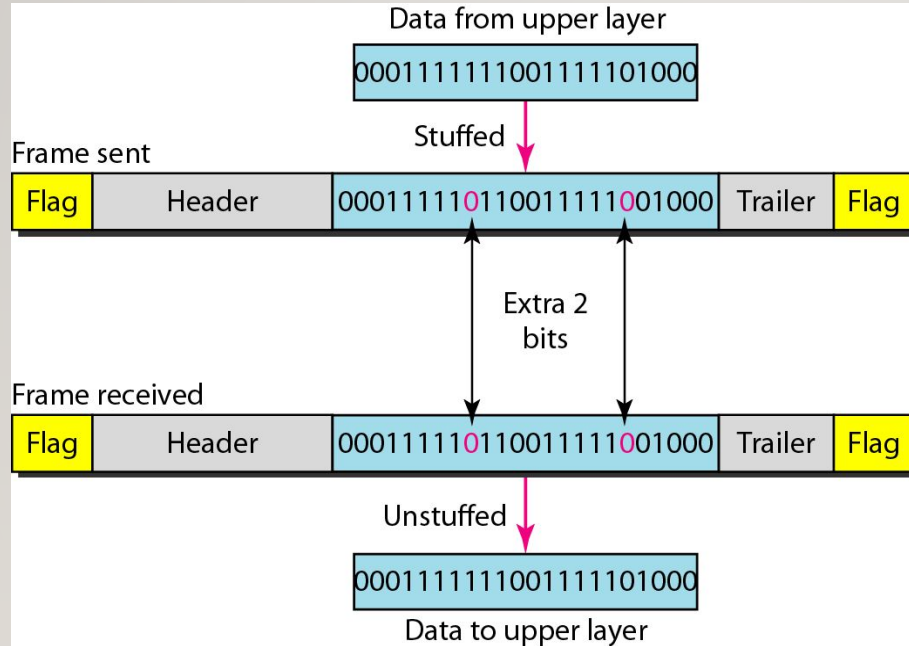
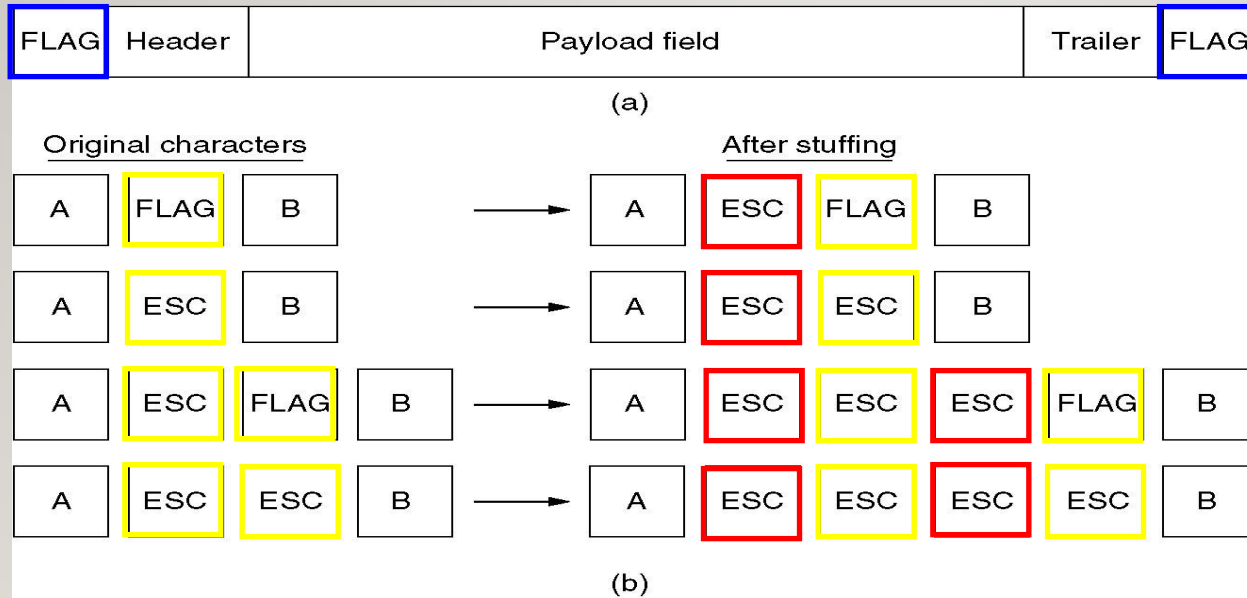


Figure 11.4 *Bit stuffing and unstuffing*



FRAMING APPROACHES (2)



(a) A frame delimited by flag bytes.

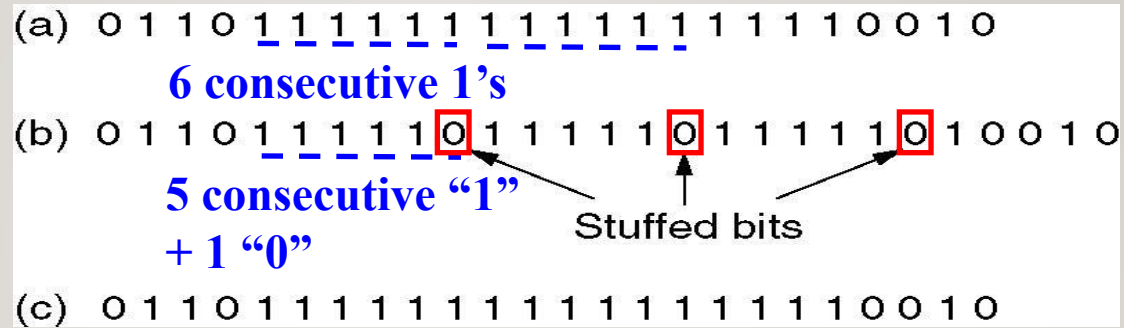
(b) Four examples of byte sequences before and after stuffing.

FRAMING APPROACHES (3)

Starting and ending flags, with bit stuffing

- Begin and end with a flag byte “01111110”
- Prevent a flag from appearing in data by bit stuffing.
- Bit stuffing: inserting 0 after five continuous bit “1” data appear.

FRAMING APPROACHES (3)



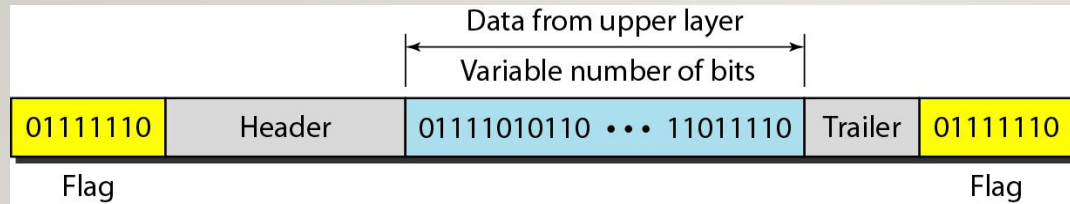
Bit stuffing

(a) The original data.

(b) The data as they appear on the line.

(c) The data as they are stored in receiver's memory after destuffing.

Figure *A frame in a bit-oriented protocol*





Note

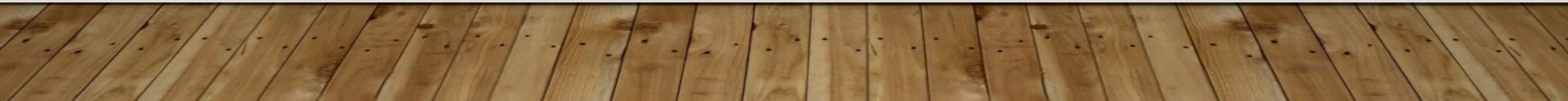
Bit stuffing is the process of adding one extra 0 whenever five consecutive 1s follow a 0 in the data, so that the receiver does not mistake the pattern 011110 for a flag.

Framing Approaches (4)

Physical Layer Coding Violations

Encoding violation is method that is used only for n/w in which Encoding on physical medium include Some sort of redundancy.

LANE Encode: Each bit of data is represented by two physical bits.



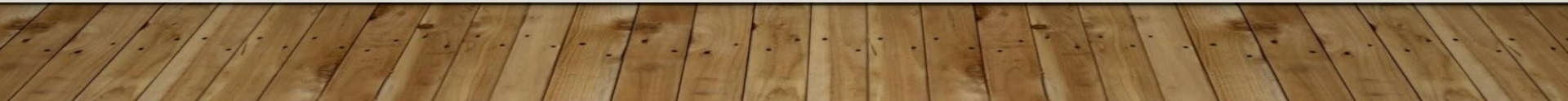
Framing Approaches (4)

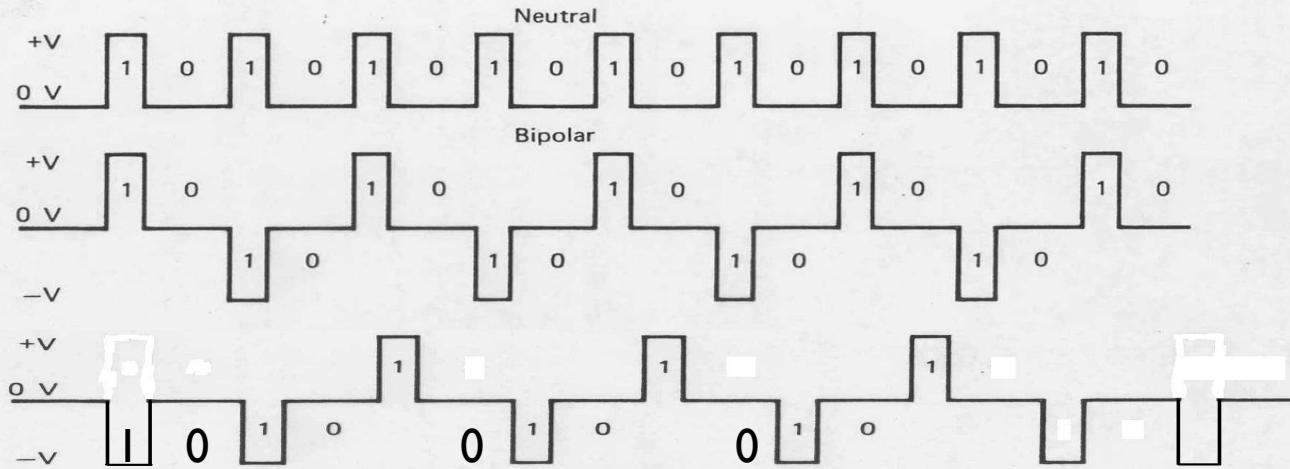
26

Physical Layer Coding Violations.

- Used when physical layer encoding contains redundancy
- Example: 01 (H), 10 (L) then 00 or 11 can be used to delimit a frame.

Note: One or more combination of the approaches may be used to provide extra protection for framing.





violati

violati

on

on

Neutral versus bipolar bit streams. (a) Alternate 1's and 0's transmitted in a neutral mode.
 (b) Equivalent in a bipolar mode.
 (c) Framing by coding violation.

Types of Errors

Errors are introduced into the binary data transmitted from the sender to the receiver due to noise during transmission. The error can be a single-bit error, multi-bit error, or burst error. Error detection methods are used to check whether the receiver has received correct data or corrupted data. And error correction is used to correct the detected errors during the transmission of data from sender to receiver.

- Single bit error :-

Only one bit in the data unit has changed.

- Multiple bit error :-

When more than one bits are corrupted.

- Burst error :-

It means that two or more bits in the data unit has changed

Single Bit Error

The following figure shows the Single bit error

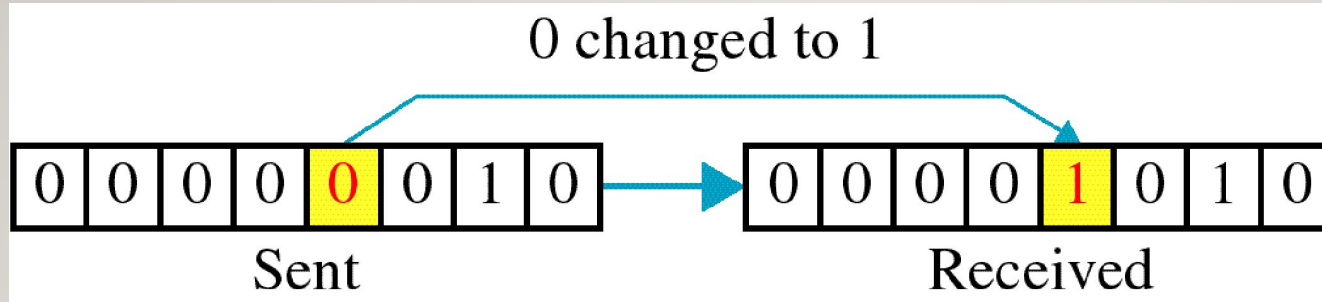


Figure 1 Single-bit error

Multiple bit error

The following figure shows the Multiple bit error

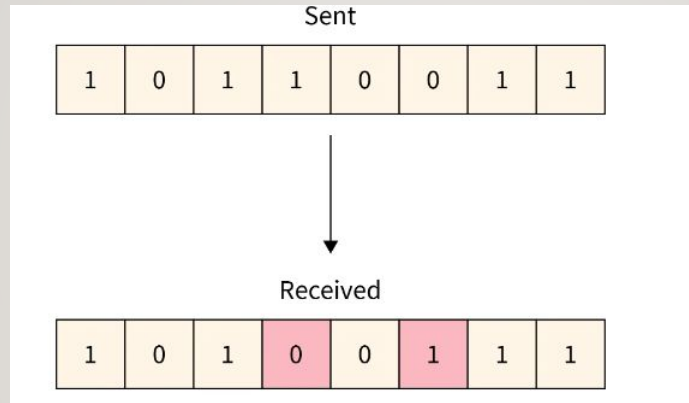


Figure 2 Multiple-bit error

Burst Error

The following figure shows the Burst error

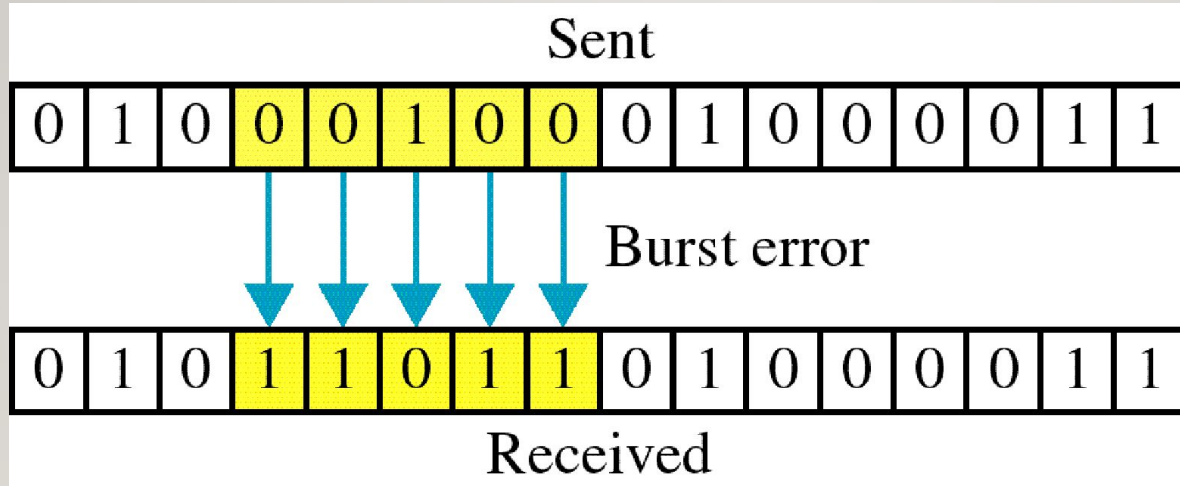


Figure 3 Burst error

Burst Error(cont..)

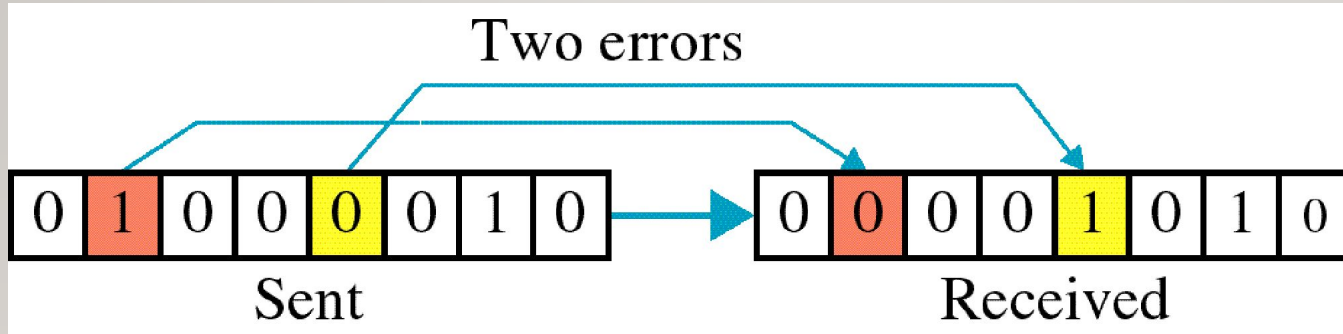


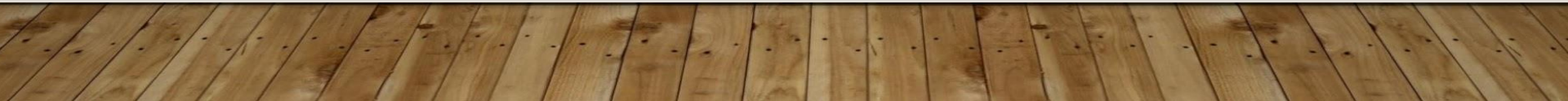
Figure 3 Two errors

The term **burst error** means that two or more bits in the data unit have changed from 1 to 0 or from 0 to 1. **Burst errors does not necessarily mean that the errors occur in consecutive bits**, the length of the burst is measured from the first corrupted bit to the last corrupted bit.

Burst Error(cont..)

Burst error is most likely to happen in serial transmission since the duration of noise is normally longer than the duration of a bit.

- The number of bits affected depends on the data rate and duration of noise.



Basic concepts

- Networks must be able to transfer data from one device to another with complete accuracy.
- Data can be corrupted during transmission.
- For reliable communication, errors must be detected and corrected.
- **Error detection and correction** are implemented either at the **data link layer** or the **transport layer** of the OSI model.
- **Single bit errors** are the **least likely** type of errors in serial data transmission because the noise must have a very short duration which is very rare. However, this kind of errors can happen in parallel transmission.

Example:

★ If data is sent at 1Mbps then each bit lasts only $1/1,000,000$ sec. or $1\mu\text{s}$.

Error Control

Two basic strategies to deal with errors:

1. Include enough redundant information to enable the receiver to deduce what the transmitted data must have been.

Error correcting codes.

2. Include only enough redundancy to allow the receiver to deduce that an error has occurred (but not which error).

Error detecting codes.

ERROR DETECTION

Error detection means to decide whether the received data is correct or not without having a copy of the original message.

Error detection **uses the concept of redundancy, which means** adding extra bits for detecting errors at the destination.



REDUNDANCY

- Instead of repeating the entire data stream, a shorter group of bits may be appended to the end of each unit.
- This technique is called Redundancy because the extra bit are redundant to the information. They are discarded as soon as the accuracy of the transmission has been determined.
 - We divide our message into blocks, each of k bits, called datawords.
 - We add r redundant bits to each block to make the length $n = k + r$. The resulting n -bit blocks are called codewords.
 - We have $2^n - 2^k$ codewords that are not used. We call these codewords invalid or illegal.

REDUNDANCY

- Instead of repeating the entire data stream, a shorter group of bits may be appended to the end of each unit.
- This technique is called Redundancy because the extra bit are redundant to the information. They are discarded as soon as the accuracy of the transmission has been determined.

There are basically four types of redundancy checks. They are:

1. VRC (Vertical Redundancy Check).
2. LRC (Longitudinal Redundancy Check).
3. CRC (Cyclical Redundancy Check).

Redundancy Checks

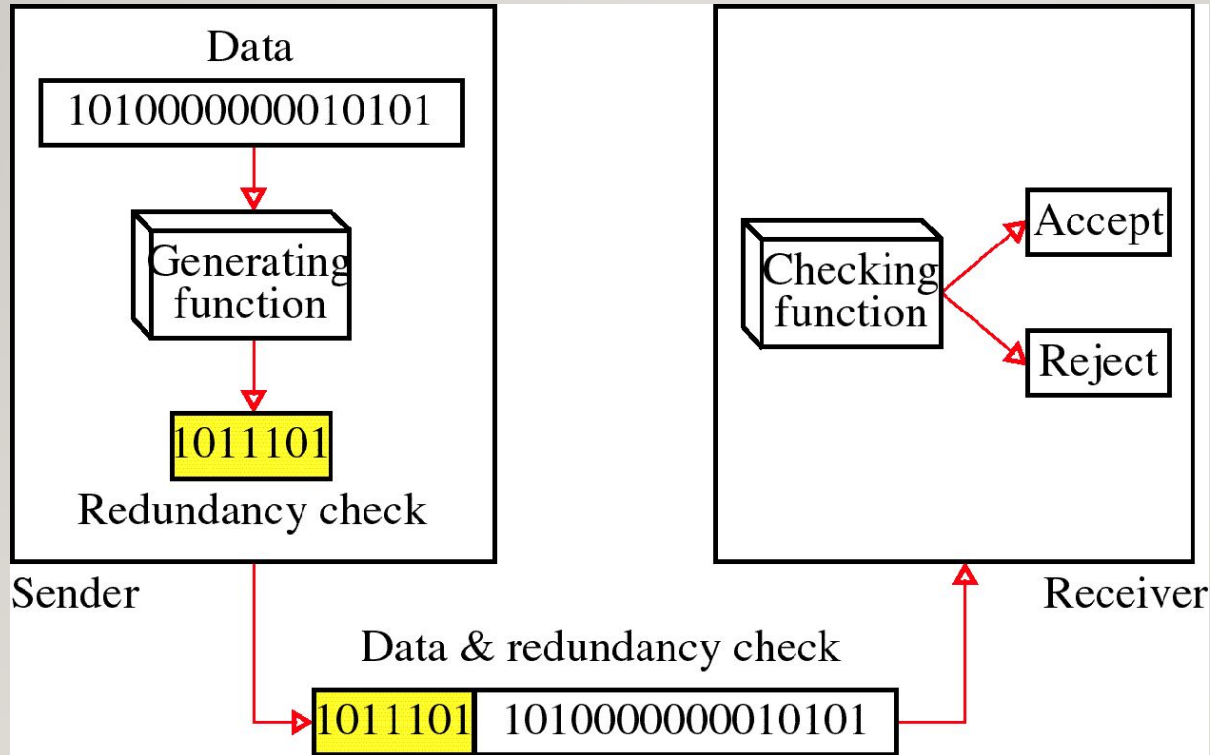
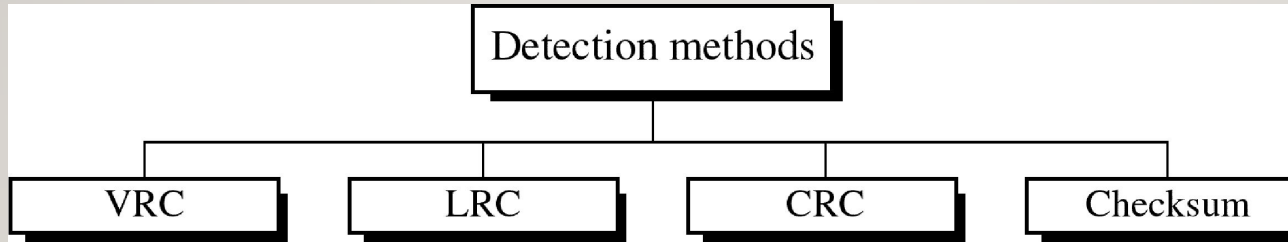


Figure 4 Redundancy Example

Redundancy Checks

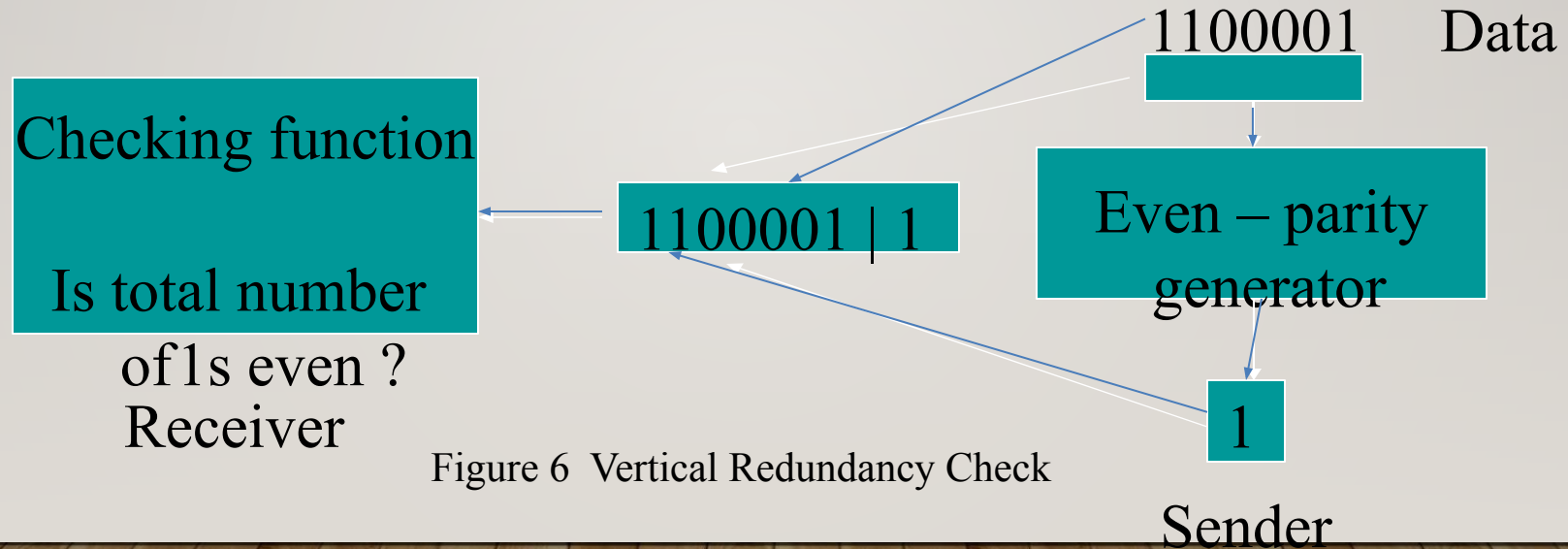
Four types of redundancy checks are used in data communications



Vertical Redundancy Check

- It is also known as parity check
- It is least expensive mechanism for error detection
- In this technique, the redundant bit called parity bit is appended to every data unit so that the total number of 1s in the unit becomes even (including parity bit)

Vertical Redundancy Check



Vertical Redundancy Check (VRC)

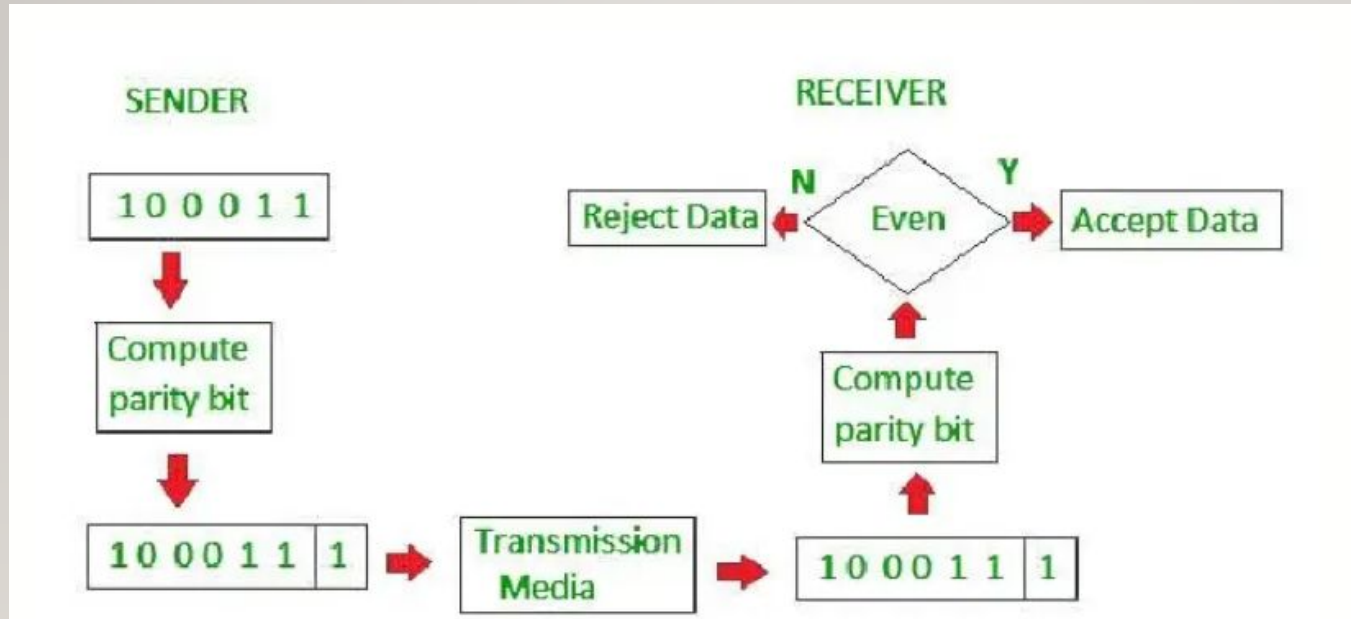


Figure 7 Vertical Redundancy Check

Parity Check

Note

In parity check, a parity bit is added to every data unit so that the total number of 1s is even (or odd for odd-parity).

Example 1

Suppose the sender wants to send the word *world*. In ASCII the five characters are coded as

1110111 1101111 1110010 1101100 1100100

The following shows the actual bits sent

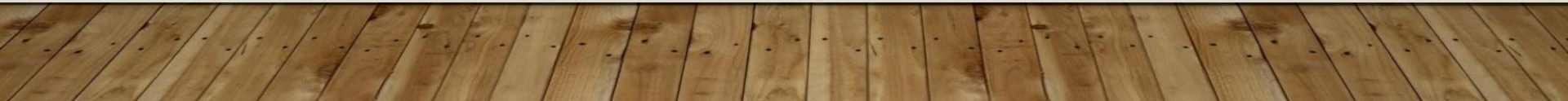
11101110 11011110 11100100 11011000 11001001

Example 2

Now suppose the word world in Example 1 is received by the receiver without being corrupted in transmission.

11101110 11011110 11100100 11011000 11001001

The receiver counts the 1s in each character and comes up with even numbers (6, 6, 4, 4, 3). The data are accepted.



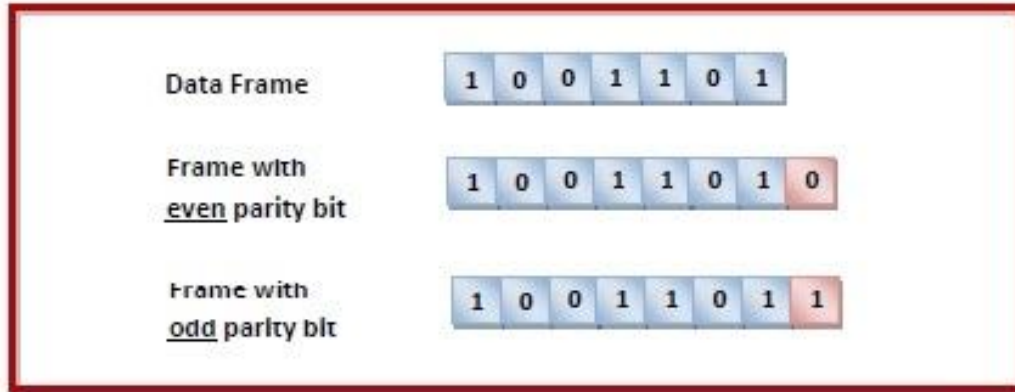
Example 3

Now suppose the word world in Example 1 is corrupted during transmission.

11111110 11011110 11101100 11011000 11001001

The receiver counts the 1s in each character and comes up with even and odd numbers (7, 6, 5, 4, 4). The receiver knows that the data are corrupted, discards them, and asks for retransmission.

- In the case of even parity: If a number of 1s is even, the parity bit value is 0. If a number of 1s is odd, the parity bit value is 1.
- In case of odd parity: If a number of 1s is odd, the parity bit value is 0. If a number of 1s is even, the parity bit value is 1.



LONGITUDINAL REDUNDANCY CHECK (LRC)

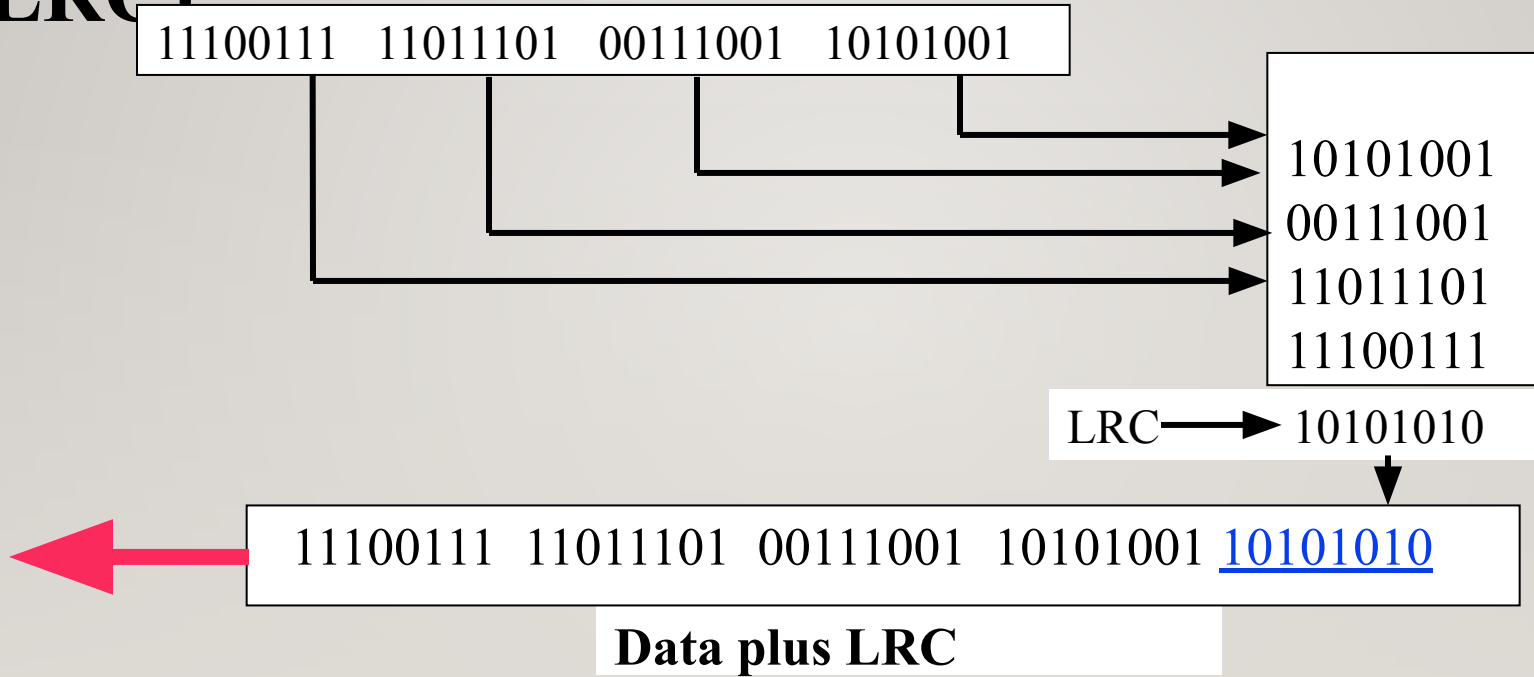


Figure 8 Longitudinal Redundancy Check

LONGITUDINAL REDUNDANCY CHECK (LRC)

Suppose the following block is sent:

10101001 00111001 11011101 11100111

10101010 (LRC)



Two-dimensional parity

In two-dimensional parity check, a block of bits is divided into rows and a redundant row of bits is added to the whole block.

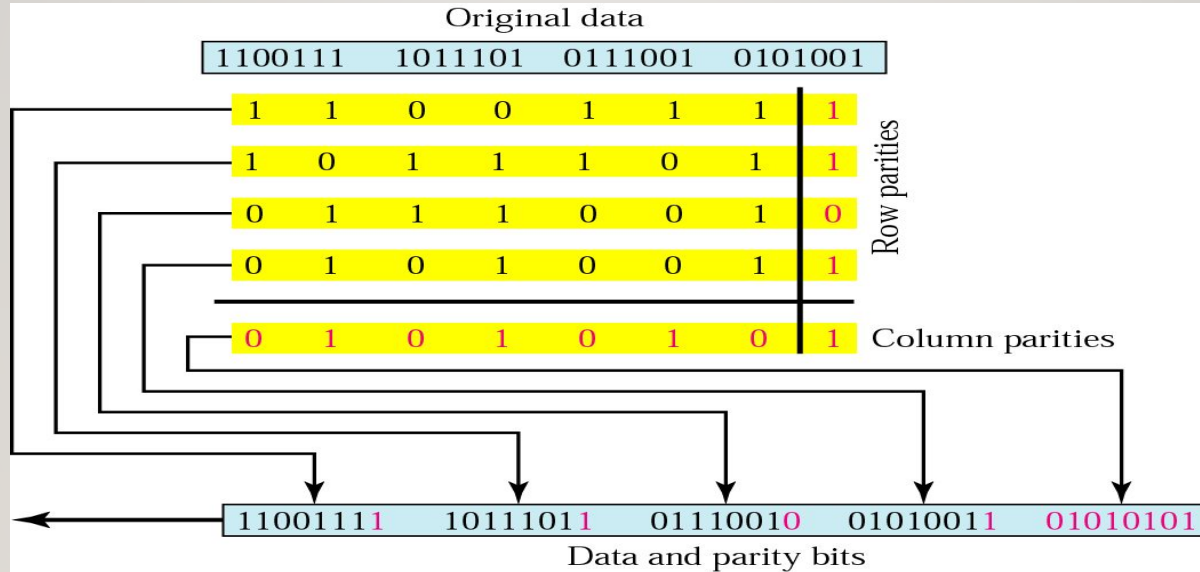


Figure 9 Two-dimensional parity-check code

Two-dimensional parity

Find the to be send with Two-Dimensional LRC (Longitudinal Redundancy Check)

Data: 1 0 1 0 1 0 1 1 1
 1 0 1 0 0 1 0 1 1 1 1 1 0 0 0 0

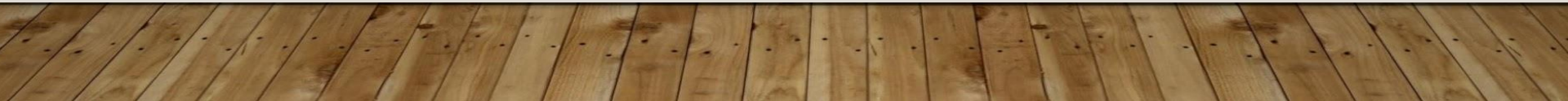
Data: 1 0 1 1 0 1 0 1 1 1 0 0 0 0 1 1
 1 0 1 1 1 0 1 0 1 0 1 1 0 0 0 0 0 1 1 0 0 0 0 1 1



Cyclical Redundancy Check (CRC)

A Cyclic Redundancy Check (CRC) is a robust, checksum-based error-detecting code used in digital networks and storage devices to identify accidental data alterations. It works by appending a remainder, calculated via modulo-2 polynomial division of the data, to the message, which the receiver verifies using the same divisor.

We can create cyclic codes to correct errors.



CRC generator and checker

Step 1: A string of 0's is appended to the data unit. It is n bits long. The number n is 1 less if number of bits in the predetermined divisor which is $n + 1$ bits.

Step 2: The newly generated data unit is divided by the divisor, using a process called as binary division. The remainder resulting from this division is the CRC.

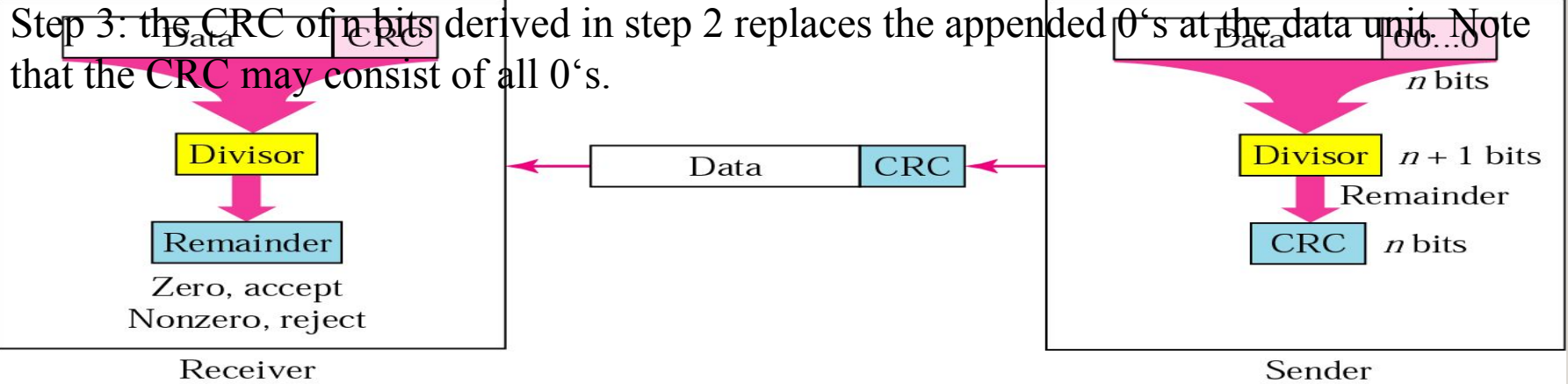


Figure 10 CRC generator and checker

CRC generator and checker

The message 100100 is to be transmitted using CRC polynomial $x^3 + x^2 + 1$ to protect it from errors. What is the message that should be transmitted.

Data : 100100

Generator Polynomial $x^3 + x^2 + 1$

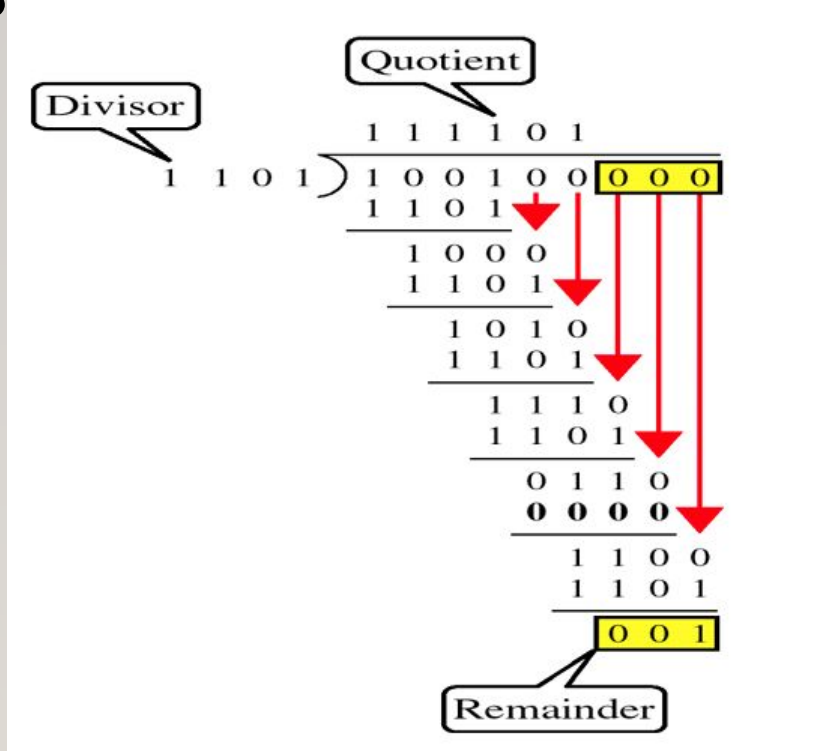
i.e. $1.x^3 + 1.x^2 + 0.x^1 + 1.x^0 = 1101$

The number n(CRC bits) is 1 less to the predetermined divisor which is n + 1 bits.

CRC bits = 000

The newly generated data unit, 100100000, is divided by the divisor, 1101

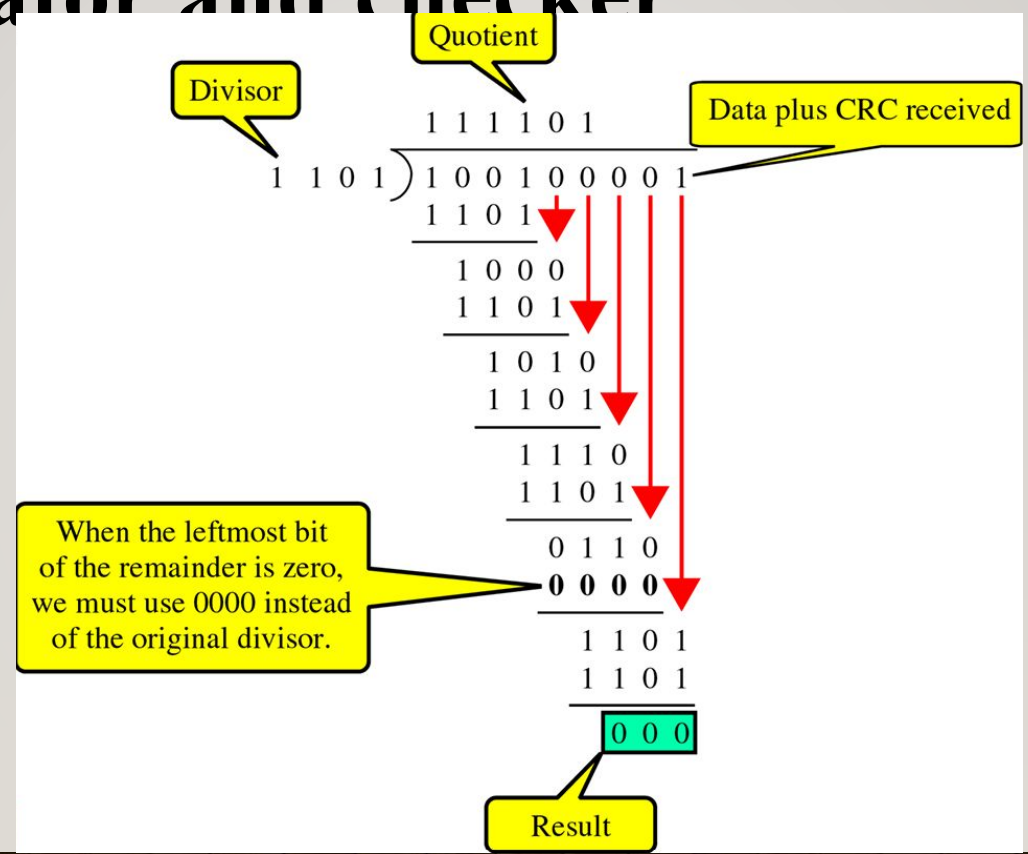
CRC generator and checker



The message that should be transmitted is 100100001

CRC generator and checker

Receiver End



CRC generator and checker

The message 11001001 is to be transmitted using CRC polynomial $x^3 + 1$ to protect it from errors. What is the message that should be transmitted.

Data : 11001001

Generator Polynomial x^3+1

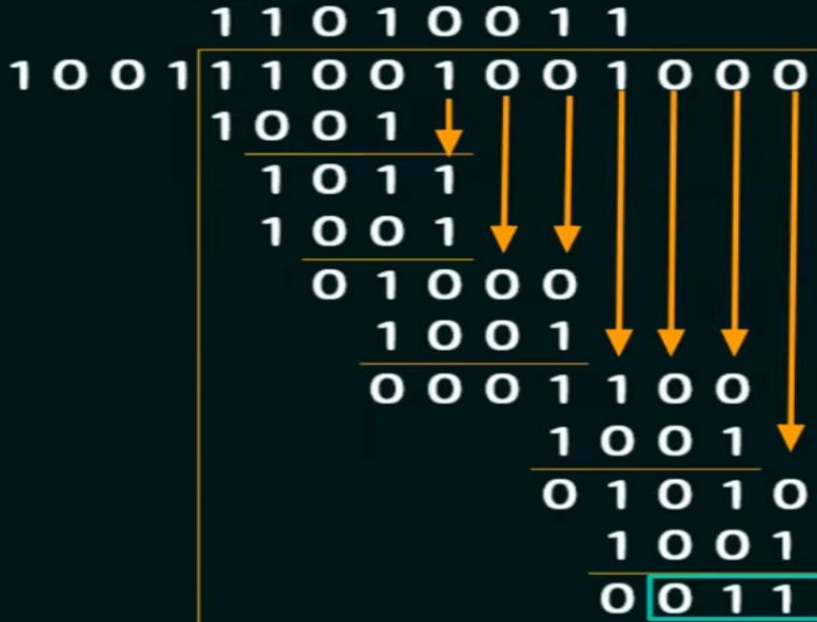
i.e. $1.x^3+ 0.x^2+ 0.x^1+ 1.x^0=1001$

CRC bits = 000

The newly generated data unit, 11001001000, is divided by the divisor, 1001. The remainder is 000. The final message to be transmitted is 11001001000.

CRC generator and checker

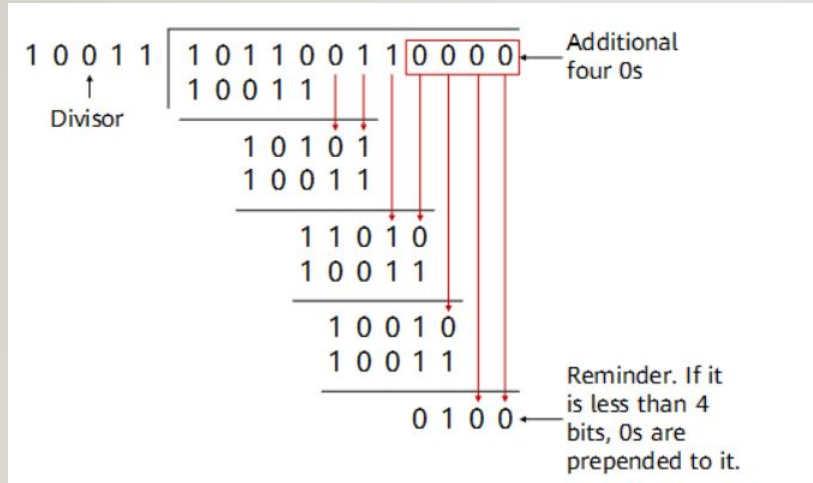
CRC GENERATION



The message that should be transmitted is 11001001011

CRC generator and checker

message 10110011 is to be transmitted using CRC polynomial $x^4 + x + 1$ to protect it from errors.



The message that should be transmitted is 101100110100

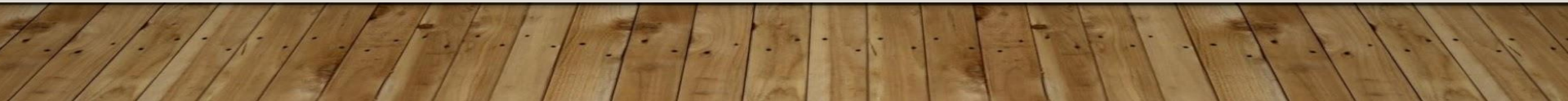
CRC generator and checker

Data 1101 is to be transmitted using CRC polynomial $x^3 + x + 1$ to protect it from errors.

The message that should be transmitted is 1101010

Data 1001 is to be transmitted using CRC polynomial $x^3 + x^2 + 1$ to protect it from errors.

The message that should be transmitted is 1001011

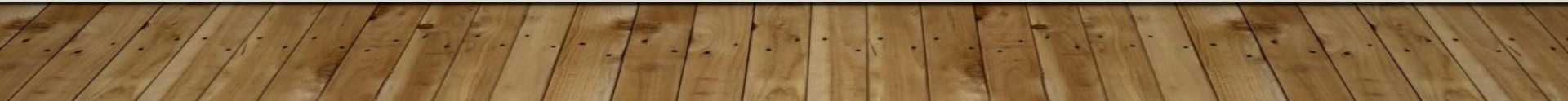


CRC

Performance:

CRC is a very effective error detection method. If the divisor is chosen according to the previously mentioned rules,

1. CRC can detect all burst errors that affect an odd number of bits.
2. CRC can detect all burst errors of length less than or equal to the degree of the polynomial.
3. CRC can detect, with a very high probability, burst errors of length greater than the degree of the polynomial.

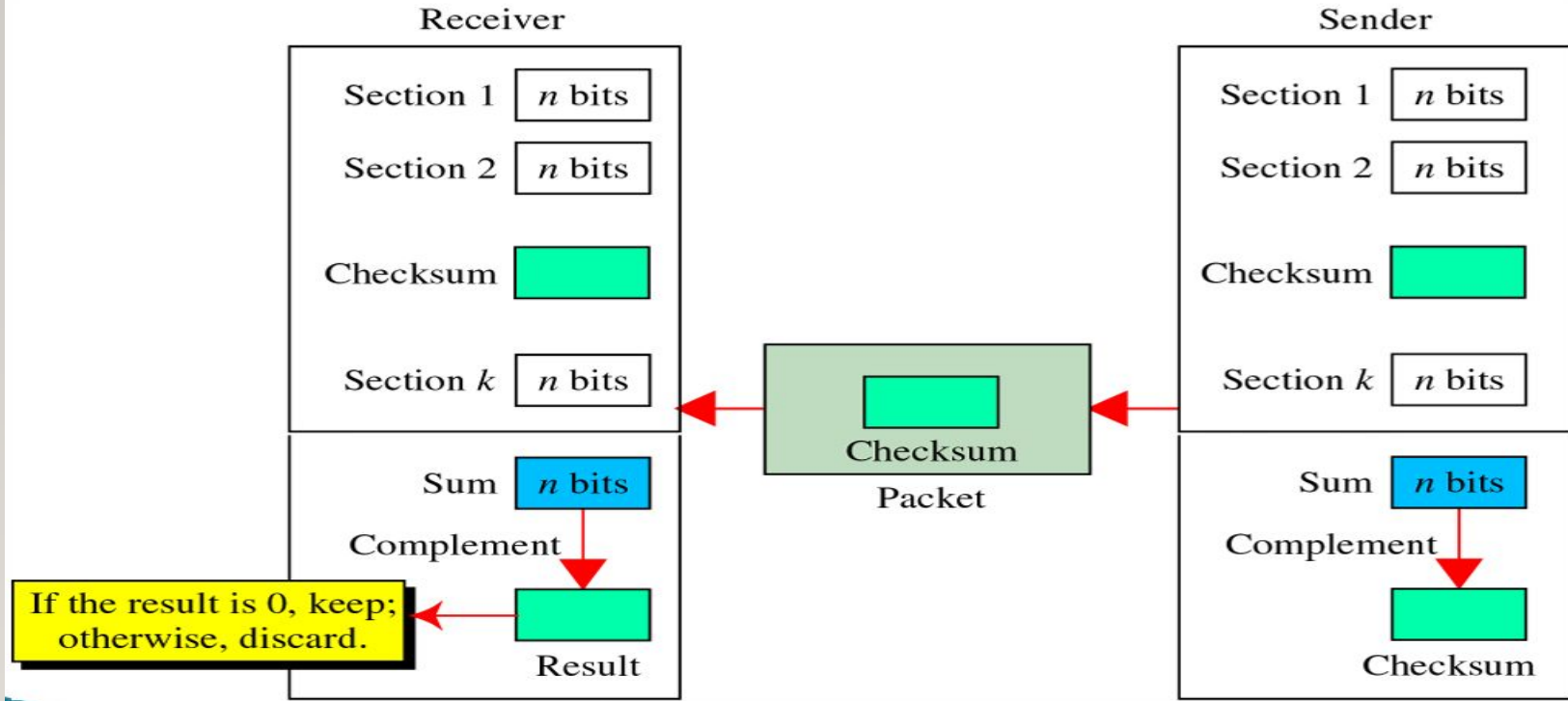


Checksum Error Detection

Checksum error detection is a technique used in computer networks to ensure data integrity during transmission. It works by using a simple mathematical sum of the data bits, calculated by the sender and verified by the receiver, to detect if any bits were altered or lost in transit.



Checksum Error Detection



Sender's steps

Note

The sender follows these steps:

- ❑ The unit is divided into k sections, each of n bits.
- ❑ All sections are added using one's complement to get the sum.
- ❑ The sum is complemented and becomes the checksum.
- ❑ The checksum is sent with the data.

Receiver's steps

Note

The receiver follows these steps:

- ❑ The unit is divided into k sections, each of n bits.**
- ❑ All sections are added using one's complement to get the sum.**
- ❑ The sum is complemented.**
- ❑ If the result is zero, the data are accepted; otherwise, rejected.**

Example

Suppose the following block of 16 bits is to be sent using a checksum of 8 bits.

10101001 00111001

The numbers are added using one's complement

10101001

00111001

Sum 11100010

Checksum **00011101**

The pattern sent is 10101001 00111001 **00011101**

Example

Now suppose the receiver receives the pattern sent in previous Example and there is no error.

10101001 00111001 00011101

When the receiver adds the three sections, it will get all 1s, which, after complementing, is all 0s and shows that there is no error.

10101001

00111001

00011101

Sum 11111111

Complement 00000000 means that the pattern is OK.

Example

Now suppose there is a burst error of length 5 that affects 4 bits.

10101111 11111001 00011101

When the receiver adds the three sections, it gets

10101111

11111001

00011101

Partial Sum **1** 11000101

Carry **1**

Sum **11000110**

Complement **00111001** the pattern is corrupted.

Example

Suppose the following block of 32 bits is to be sent using a checksum of 8 bits.

10011001 11100010 00100100 10000100

Sum of first 2 blocks.

10011001

Carry 1

~~11100010~~
~~01111011~~

01111100

Add 3rd block

~~00100100~~
10100000

Add 4th block

Carry 1

~~10000100~~
~~00100100~~

1

Sum

00100101

1's Complement

11011010

Example

10011001

11100010

00100100

10000100

11011010

Sum of first 2 blocks.

10011001

Carry

1

~~11100010~~
01111011

01111100

Add 3rd block

~~00100100~~
10100000

Add 4th block

Carry

1

~~10000100~~
00100100

Add 5th block

00100101
~~11011010~~

Sum

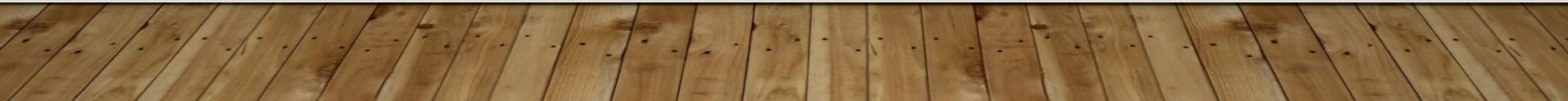
11111111

1's Complement

00000000

Difference between VRC and LRC

Sr.No.	Vertical Redundancy Check (VRC)	Longitudinal Redundancy Check (LRC)
1.	It stands for Vertical Redundancy Check.	It stands for Longitudinal Redundancy Check.
2.	In this redundant bit called parity bit is added to each data unit.	In this redundant row of bits is added to the whole block.
3.	VRC can detect single bit errors.	LRC can detect burst errors.
4.	It is also known as parity checker.	It is also known as 2-D parity checker.
5.	The advantage of using VRC is that it can checks all single bit errors but can check odd parity only in the case of change of odd bits.	The advantage of using LRC over VRC is that it can check all the burst errors.
6.	It is not capable of checking the burst error in case of change of bits is even.	If two bits in data unit is damaged and also in other data unit the same bits are damaged at same position, then it is not capable of detecting such kind of error.



Correction

Backward Error Correction

- Retransmission

Forward Error Correction

- Hamming Code



HAMMING CODE

Data and redundancy bits

Number of data bits m	Number of redundancy bits r	Total bits $m + r$
1	2	3
2	3	5
3	3	6
4	3	7
5	4	9
6	4	10
7	4	11

$$2^r \geq m + r + 1$$

Positions of redundancy bits in Hamming code

11	10	9	8	7	6	5	4	3	2	1
d	d	d	r_8	d	d	d	r_4	d	r_2	r_1

$$2^3, 2^2, 2^1, 2^0$$

Redundancy bits calculation

d7 d6 d5 **r8** d4 d3 d2 **r4** d1 **r2** **r1**

R1 will take care of these bits

d7 d6 d5 **r8** d4 d3 d2 **r4** d1 **r2** **r1**

R2 will take care of these bits

d7 d6 d5 **r8** d4 d3 d2 **r4** d1 **r2** **r1**

R4 will take care of these bits

d7 d6 d5 **r8** d4 d3 d2 **r4** d1 **r2** **r1**

R8 will take care of these bits

d7 d6 d5 **r8** d4 d3 d2 **r4** d1 **r2** **r1**

Redundancy bits calculation

Data: 1001101

1	0	0	r8	1	1	0	r4	1	r2	r1
---	---	---	----	---	---	---	----	---	----	----

R1 will take care of these bits

1	0	0	r8	1	1	0	r4	1	r2	1
---	---	---	----	---	---	---	----	---	----	---

R2 will take care of these bits

1	0	0	r8	1	1	0	r4	1	0	1
---	---	---	----	---	---	---	----	---	---	---

R4 will take care of these bits

1	0	0	r8	1	1	0	0	1	0	1
---	---	---	----	---	---	---	---	---	---	---

R8 will take care of these bits

1	0	0	1	1	1	0	0	1	0	1
---	---	---	---	---	---	---	---	---	---	---

Error detection using Hamming code

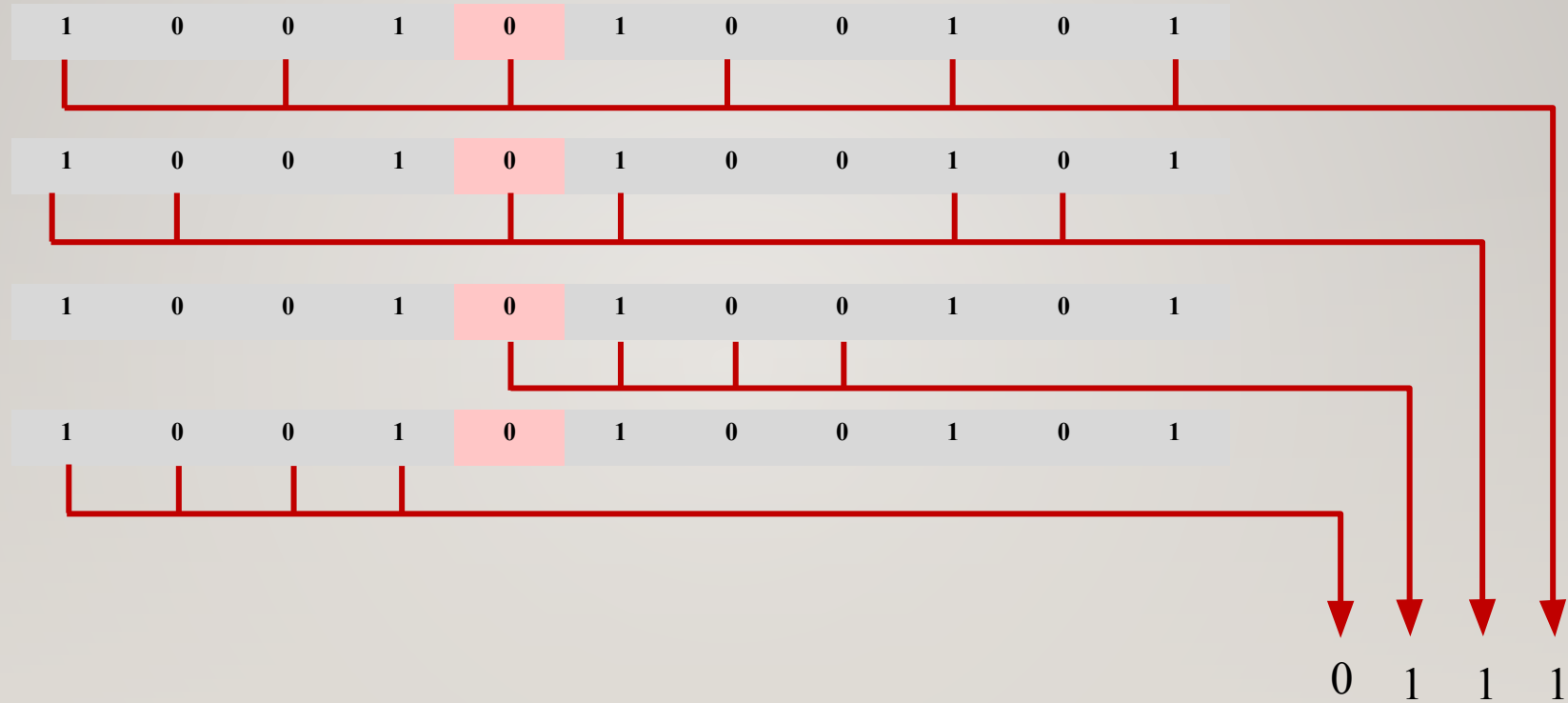


Figure 13-Example of Hamming Code

Error detection using Hamming code

Data: 0110

Message: 0110011

Received: 1110011



Burst error correction Example

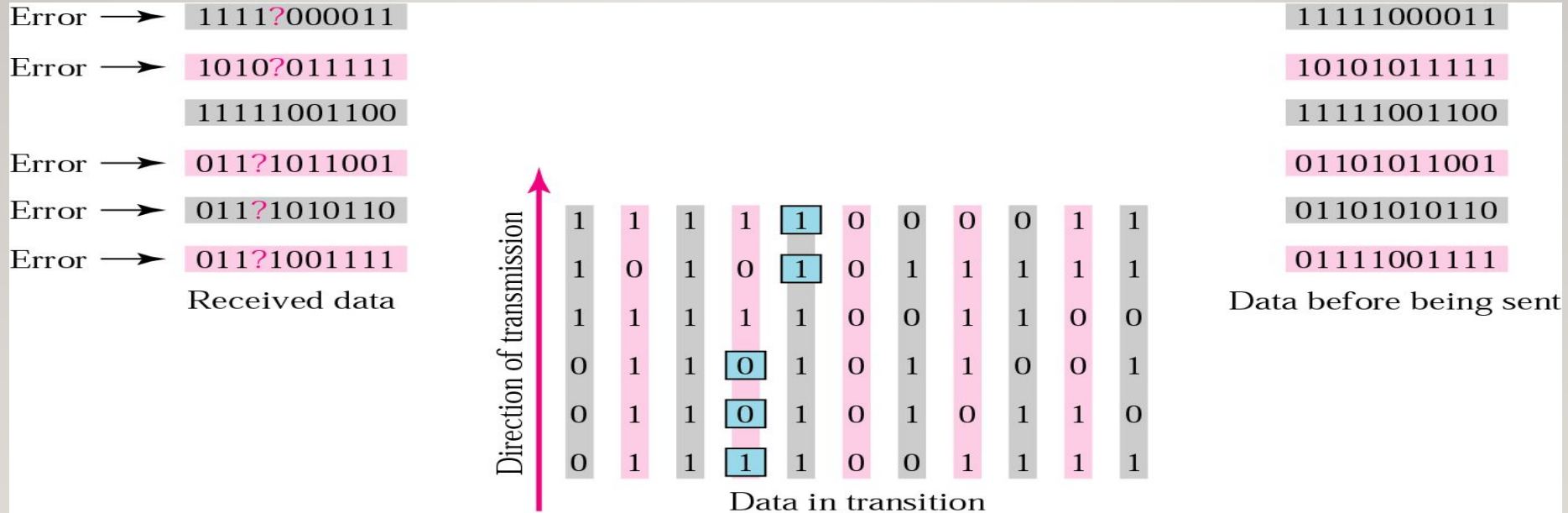


Figure 14 Burst Error Correction Example

Consider 5 32 bit block of data 11100111 11011101 00111001 10101001 that has to be transmitted. If Longitudinal Redundancy Check is used what is the transmitted bit stream?

Answer:

11100111 11011101 00111001 10101001 10101010.



A receiver gets data segments 11011011, 10101010, and checksum 01110001. Verify the data.

Answer:

Error in the transmitted data

A receiver gets the frame 1101011011011 and uses divisor 1011. Is there an error?

Answer:

Error in the transmitted data

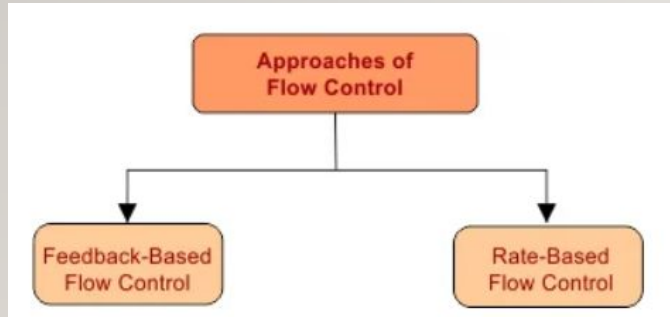
Flow Control

Flow control in the data link layer is essential because different nodes in a network may have varying capacities and transmission speeds. To ensure smooth and successful communication between these nodes, effective and reliable control of the data flow is necessary.

- Flow control coordinates the amount of data that can be sent before receiving an acknowledgment
- One of the most important duties of the data link layer.
- In most protocols, flow control is a set of procedures that tells the sender how much data it can transmit before it must wait for an acknowledgment from the receiver.

Approaches To Flow Control

Flow Control can be handled through the following approaches.

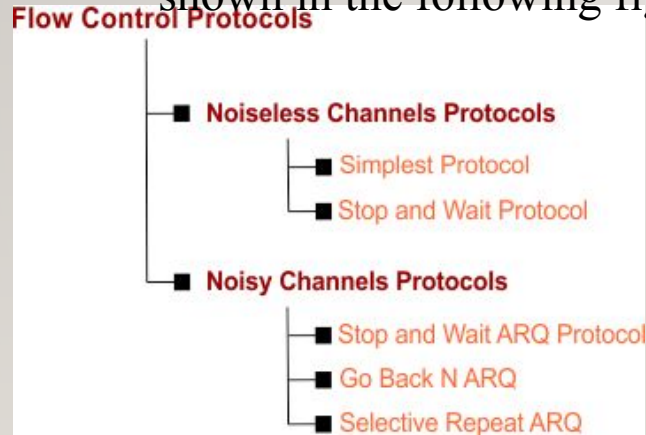


1. Feedback-based Flow Control: In these protocols, the sender sends frames to the receiver and then waits for an acknowledgment signal from the receiver. This approach is used in the data link layer.

2. Rate-based Flow Control: These kinds of protocols do not use any feedback or acknowledgment signals. These protocols have built-in mechanisms to limit or restrict the rate of data transmission from sender to receiver. This approach is mostly used in the network layer and the transport layer.

Flow Control Protocols in Data Link Layer

There are basically two types of techniques used to control the flow of data. One for the Noiseless channel and the other for the Noisy Channel, as shown in the following figure.



Noiseless Channels (Idealized)

A hypothetical, perfect channel where no frames are lost, damaged, or corrupted.

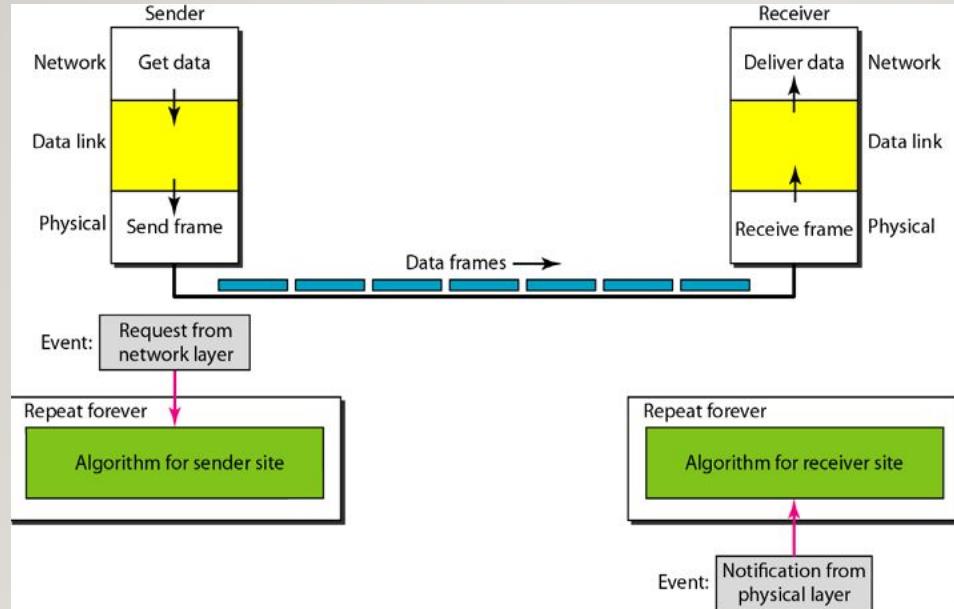
Noisy Channels (Realistic)

Definition: Real-world channels affected by electromagnetic interference, attenuation, and noise causing signal degradation

Simplest Protocol

- It is a unidirectional protocol in which data frames are traveling in only one direction-from the sender to receiver.
- The receiver can immediately handle any frame it receives with a processing time that is small enough to be negligible.
- The data link layer of the receiver immediately removes the header from the frame and hands the data packet to its network layer, which can also accept the

Simplest Protocol



Flow Control Protocols in Data Link Layer

Main Flow Control Techniques

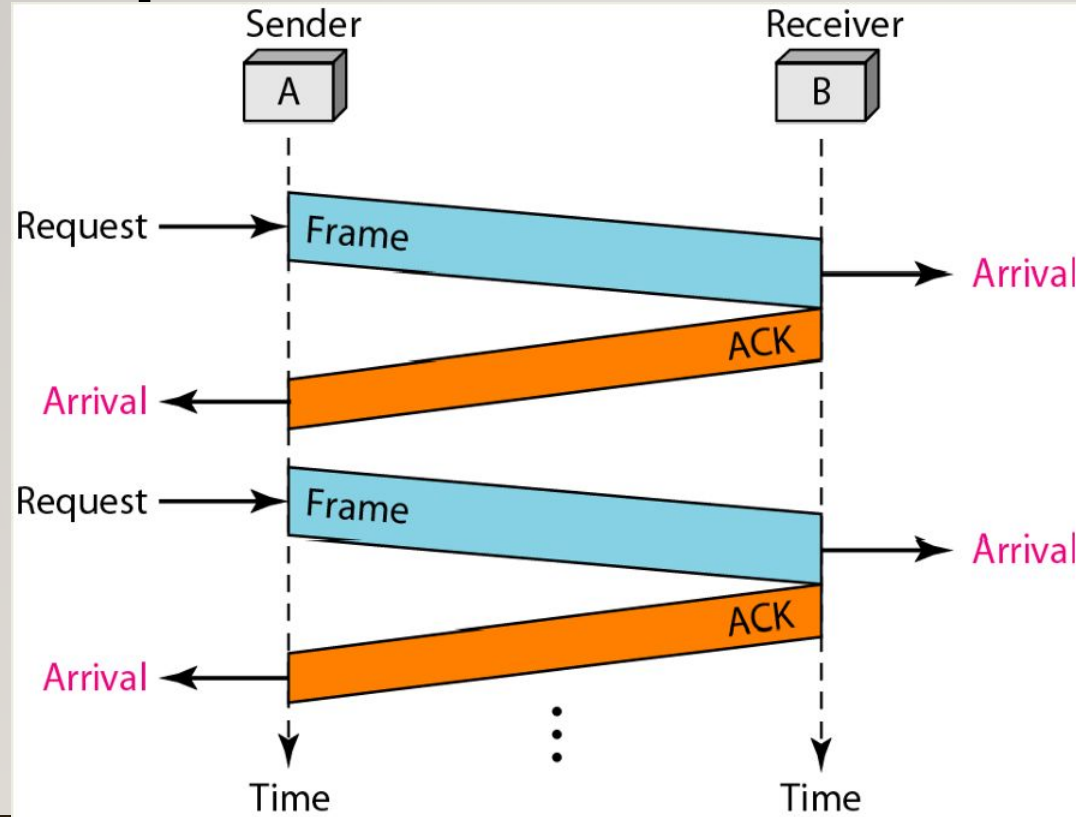
- **Stop-and-Wait**: The sender transmits one frame, then waits for an acknowledgment (ACK) from the receiver before sending the next, which is simple but inefficient.
- **Sliding Window**: The sender can transmit multiple frames before receiving an acknowledgement, allowing for higher efficiency and better utilization of network capacity.

Stop-and-wait flow control

In the stop-and-wait approach, the sender segments the data into frames and then transmits one frame at a time to the receiver, which responds to each frame with an ACK message. This process occurs through the following steps:

- The sender transmits a data frame to the receiver.
- The sender waits for the receiver to respond.
- Upon receiving the frame, the receiver transmits an ACK to the sender.
- Upon receiving the ACK, the sender sends the next frame to the receiver and waits for the next ACK. If the sender does not receive an ACK within a defined time limit, known as a timeout, the sender retransmits the same frame.
- The process continues until the sender has finished transmitting all the data to the receiver.

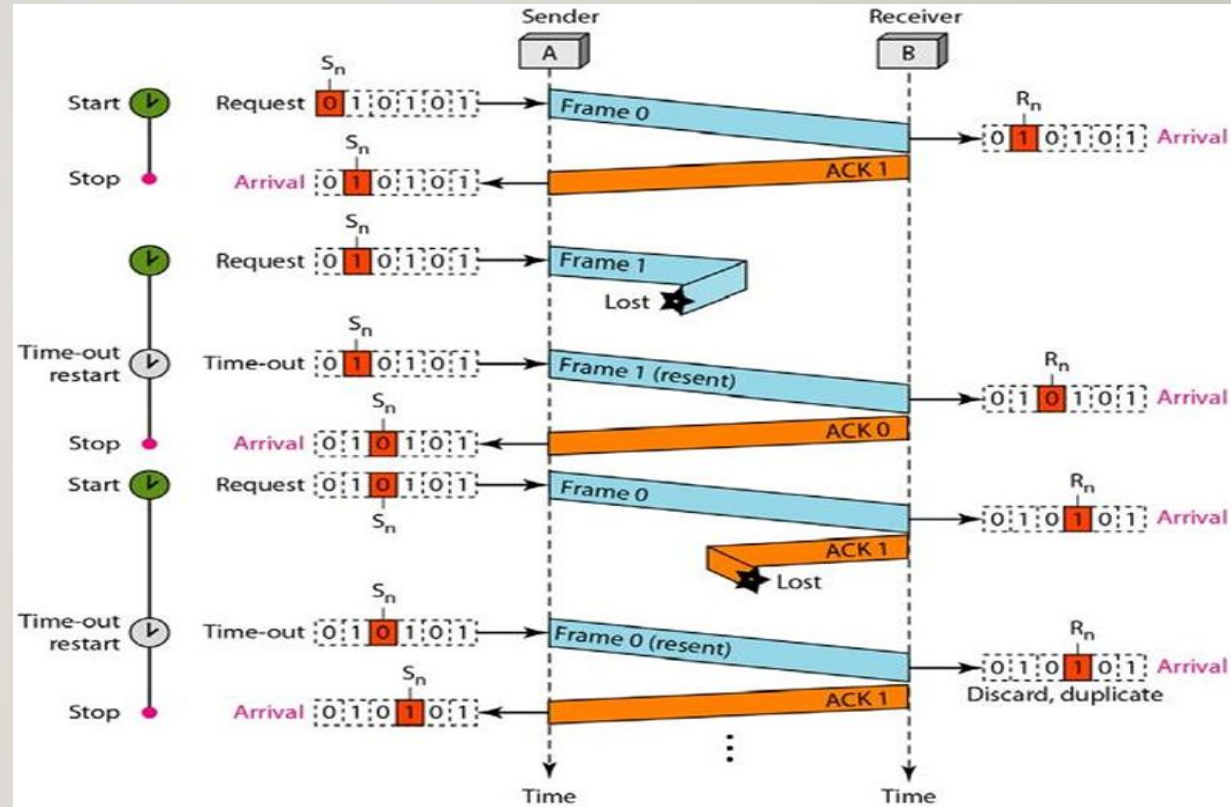
Stop-and-wait flow control



Stop-and-wait flow control

The sender keeps a copy of the sent frame. At the same time, it starts a timer.

- If the timer expires and there is no ACK for the sent frame, the frame is resent, the copy is held, and the timer is restarted.
- The protocol uses the stop-and-wait mechanism, there is only one specific frame that needs an ACK even though several copies of the same frame



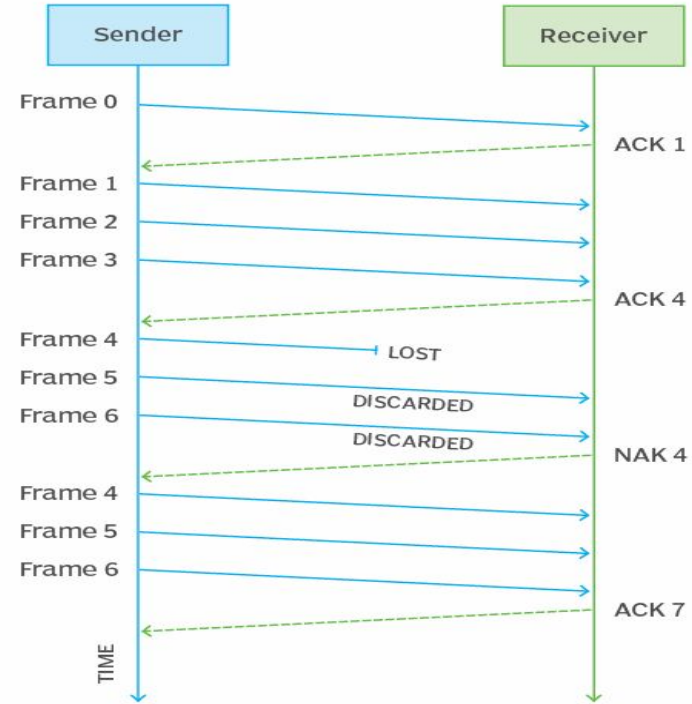
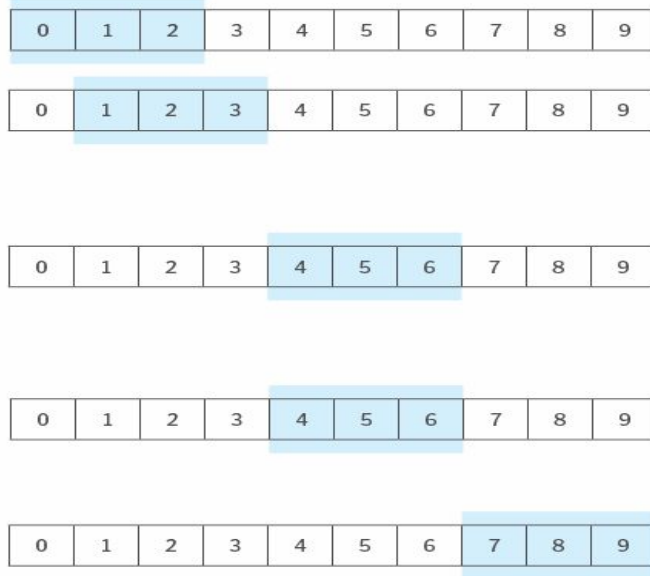
Sliding window (Go-back-N) flow control

The sliding window approach addresses many of the issues that come with stop and wait because the sender can transmit multiple frames at once without having to wait for an ACK for each frame. However, this approach also comes with additional complexity.

When first connecting, the sender and receiver establish a window that determines the maximum number of frames the sender can transmit at a time. During the transmission, the sender and receiver must carefully track which frames have been sent and received to ensure that all the data reaches its destination and is reassembled in the correct order.

Sliding window flow control can be implemented using one of two approaches: Go-Back-N and Selective Repeat. With the Go-Back-N approach, the sender can send one or more frames but never more frames than the window allows. As the receiver acknowledges the frames, the sender moves to the next batch, or window, of frames

Sliding window (Go-back-N) flow control



Sliding window (Go-back-N) flow control

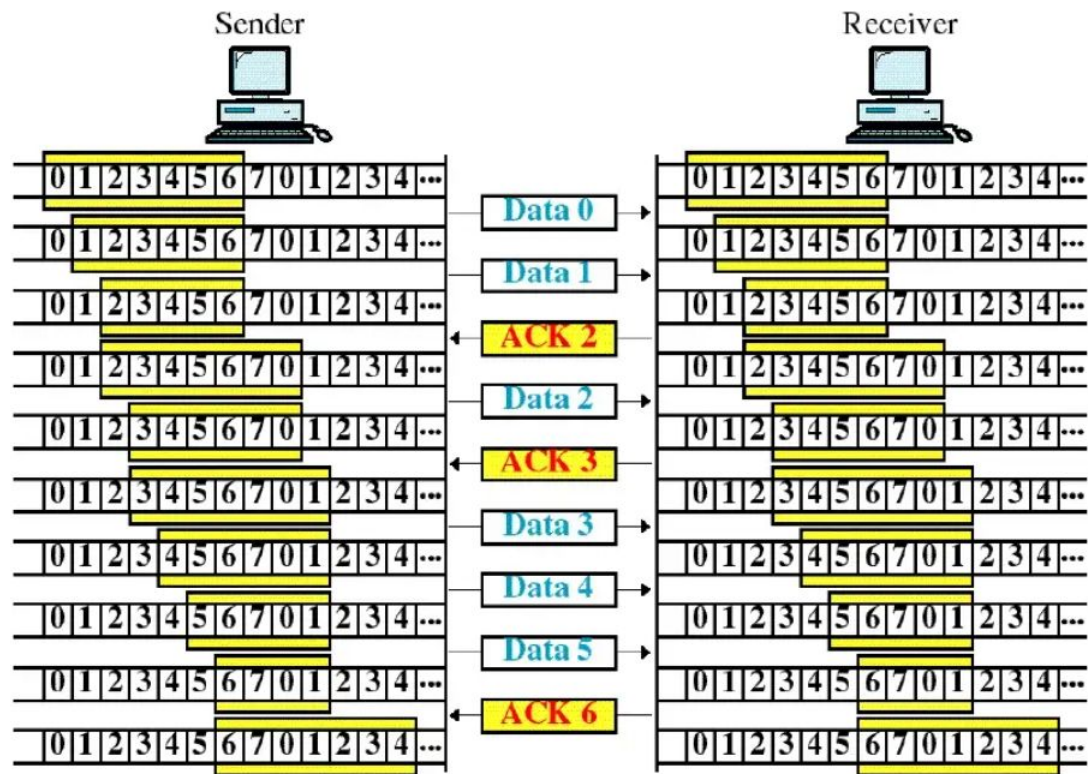
Figure shows an example of how Go-Back-N works. In this case, the window consists of only three frames -- initially, Frames 0 through 2. The sender begins by transmitting Frame 0 to the receiver. Upon receiving Frame 0, the receiver sends an ACK that specifies the next frame to send (Frame 1), rather than specifying the frame that has just been received. When the sender receives ACK 1, it moves the window over by one position, dropping Frame 0 and adding Frame 3. The sender then transmits Frames 1, 2 and 3, which represent the window's entire contents. The sender does not necessarily need to send Frame 0 first, followed by Frames 1 through 3. This is illustrated in Figure 2 to demonstrate how the process works.

Upon receiving the three frames, the receiver sends a cumulative ACK that specifies the next frame to send, which is Frame 4. The ACK indicates that the receiver now has all the preceding frames (0 through 3).

Sliding window (Go-back-N) flow control

When the sender receives ACK 4, it adjusts the window so that it now includes Frames 4 through 6 and then transmits those frames. This time, however, Frame 4 gets lost in the transmission, while Frames 5 and 6 reach their destination. Upon receiving Frame 5, the receiver detects that Frame 4 is missing and sends a negative acknowledgement (NAK) that specifies Frame 4. At the same time, the receiver discards Frames 5 and 6.

When the sender receives the NAK, it retransmits Frames 4 through 6 and waits for the ACK. The frames arrive with no errors the second time around, so the receiver returns an ACK indicating that the sender can now transmit Frame 7. The sender adjusts the window accordingly and transmits the next set of frames, starting with Frame 7.



Sliding window (Selective Repeat) flow control

The Selective Repeat approach is similar to Go-Back-N. The primary difference is that Selective Repeat does not retransmit the entire window if there is an error, only the individual frame in dispute. Selective Repeat does not support cumulative ACK messages like Go-Back-N, so each ACK is specific to the frame that was just received, which is what enables the sender to identify the precise frame that needs to be retransmitted.

Figure illustrates an example of the Selective Repeat process. After transmitting Frame 0, the sender receives an ACK, so it transmits Frames 1 through 3 and receives an ACK for each one. The sender then transmits Frames 4 through 6. When Frames 5 and 6 arrive at the receiver, but not Frame 4, the receiver sends ACK 5 and ACK 6, along with NAK 4. The sender responds to the NAK by retransmitting Frame 4. Upon receiving Frame 4, the receiver

Sliding window (Selective Repeat) flow control

Sliding window flow control (Selective Repeat)

